ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл холбооны технологийн сургууль

ЛАБОРАТОРИЙН АЖЛЫН ТАЙЛАН

Хиймэл оюуны үндэс
2026-2027 оны хичээлийн жилийн намар

Лабораторийн ажлын нэр: Лабораторийн ажил №3
Нийлмэл өгөгдлийн төрөл, файлтай ажиллах, класс, дараалал, стек

Хичээл заасан багш: .....

Лабораторийн ажил гүйцэтгэсэн: Оюутан: Ц.Номинжин - B241940137

Авбал зохих нийт оноо:	**15 оноо**		
Гүйцэтгэлийн шалгуур	**Үнэлгээний эзлэх хувь**	**CLOs хамаарал**	**Багшийн үнэлгээ**
Хичээлийн ирц/оролцоо	10%	7,8	
Даалгавар 1	20%	7	
Даалгавар 1	20%	8	
Даалгавар 2	30%	9	
Лабораторийн тайлан бичилт, хамгаалалт	10%+10%	10	
Үнэлгээний нийт хувь			
Нийт авсан оноо			

ТОЙМ

Лабораторийн ажлын зорилго нь Python хэл дээр нийлмэл өгөгдлийн төрлүүд (list, tuple, set, dictionary), файлтай ажиллах горимууд, класс, удамшил, deque дээр суурилсан дараалал ба стекийг бодлогод хэрэглэн туршихад оршсон. Программуудыг IDLE орчинд бичиж ажиллуулан, гаралтыг дэлгэцийн зургаар баримтжуулсан. Лаб 3.1-д сондгой индекстэй сөрөг элементүүдийн үржвэр (36), тэгш ба сондгой тоонуудын эрэмбэлсэн жагсаалт, 12 нэр томьёотой Монгол–Англи толь, санамсаргүй тоон жагсаалт, өнгө ба үнээр хайдаг жимсний программ бичигдсэн. Лаб 3.2-т open() функцийн 'w', 'a', 'r' горимууд туршигдаж, reviews.txt файлын сэтгэгдэлд түлхүүр үгэнд суурилсан мэдрэмжийн шинжилгээ хийгдэн үр дүн results.txt-д хадгалагдсан. 'a' горимоор 3 сэтгэгдэл нэмж 9 сэтгэгдлийг дахин шинжилсэн бөгөөд 7 нь (77.8 %) зөв ангилагдсан; «decent» үгийг эерэг жагсаалтад оруулснаас нэг сэтгэгдэл, үгүйсгэл агуулсан «I am not happy with this service.» сэтгэгдэл тус тус буруу ангилагдсан. Лаб 3.3-т Task, DataProcessingTask, ModelEvaluationTask, TaskQueue, TaskHistory ангиуд бичигдэж, 4 даалгаврын нэг нь цуцлагдан, нэг нь хоосон замаас болж бүтэлгүйтсэн бол үлдсэн нь тэргүүлэх зэргийн дагуу (T102, T104, T103) боловсруулагдсан. Ижил зэрэгтэй даалгаврууд ирсэн дарааллаа хадгалсан нь sorted() функцийн тогтвортой байдлыг харуулав. Ажлаар түлхүүр үгэнд суурилсан шинжилгээ үгүйсгэлийг ялгаж чаддаггүй, харин deque нь дараалал ба стекийг хялбар хэрэгжүүлдэг нь тогтоогдов.

ГАРЧИГ

ТОЙМ ii
ЗУРГИЙН ЖАГСААЛТ iv
ХҮСНЭГТИЙН ЖАГСААЛТ v
1. АЖЛЫН ЗОРИЛГО БА ОНОЛЫН ОЙЛГОЛТ 1
1.1. Ажлын зорилго 1
1.2. List 1
1.3. Tuple 1
1.4. Set 1
1.5. Dictionary 1
1.6. Файлтай ажиллах ба мэдрэмжийн шинжилгээ 2
1.7. Класс ба удамшил 3
1.8. Дараалал ба стек 3
2. ХЭРЭГЛЭГДЭХҮҮН 4
3. ХЭМЖИЛТ, ТУРШИЛТ ХИЙСЭН ДАРААЛАЛ 4
3.1. Лаб 3.1: нийлмэл өгөгдлийн төрлүүд 5
3.2. Лаб 3.2: файлтай ажиллах 5
3.3. Лаб 3.3: класс, удамшил, дараалал, стек 6
4. ХЭМЖИЛТ, ТУРШИЛТЫН ҮР ДҮНГҮҮД 6
4.1. Лаб 3.1-ийн үр дүн 6
4.2. Лаб 3.2-ийн үр дүн 8
4.3. Лаб 3.3-ын үр дүн 10
5. ХЭЛЭЛЦҮҮЛЭГ 12
5.1. Лаб 3.1-ийн үр дүнгийн шинжилгээ 12
5.2. Лаб 3.2-ийн үр дүнгийн шинжилгээ 14
5.3. Лаб 3.3-ын үр дүнгийн шинжилгээ 16
5.4. Зорилтод хүрсэн байдал 17
6. ДҮГНЭЛТ 18
7. АШИГЛАСАН МАТЕРИАЛ 19
8. ХАВСРАЛТ 20
ХАВСРАЛТ А. Лаб 3.1-ийн эх код 20
ХАВСРАЛТ Б. Лаб 3.2-ын эх код 22
ХАВСРАЛТ В. Лаб 3.3-ын эх код 25

ЗУРГИЙН ЖАГСААЛТ

Зураг 1.1. Лаб 3.3-ын программын ангиудын бүтэц (UML диаграм) 3
Зураг 1.2. Дараалал (FIFO) ба стек (LIFO)-ийн ажиллагааны схем 4
Зураг 4.1. Жишээ 1: `sum_list` функцийн гаралт 7
Зураг 4.2. Жишээ 2: номын сангийн программын гаралт (анхны нэрс) 7
Зураг 4.3. Жишээ 2: номын нэрийг монгол нэрээр сольсон гаралт 7
Зураг 4.4. Бодлого 1: сондгой индекстэй сөрөг элементүүдийн үржвэр 7
Зураг 4.5. Бодлого 2: тэгш ба сондгой тоонуудын жагсаалт 7
Зураг 4.6. Бодлого 3: Монгол–Англи нэр томьёоны толь 7
Зураг 4.7. Бодлого 4: санамсаргүй бүхэл тоонуудын жагсаалт 7
Зураг 4.8. Бодлого 5: жимсний мэдээлэл боловсруулах программ 8
Зураг 4.9. Жишээ 1: `'w'`, `'a'`, `'r'` горимоор бичих, унших 8
Зураг 4.10. Жишээ 2: шинэ мөрийн тэмдэгтэй болон тэмдэггүй нэмэх 8
Зураг 4.11. Жишээ 3: `with` хэллэг ашиглан бичих, унших 9
Зураг 4.12. Бодлого 1: `reviews.txt` файл үүсгэсэн байдал 9
Зураг 4.13. Мэдрэмжийн шинжилгээний жишээ программын гаралт ба `results.txt` 9
Зураг 4.14. Бодлого 2: мэдрэмжийн шинжилгээний гаралт ба `results.txt` 9
Зураг 4.15. Бодлого 3: `'a'` горимоор нэмсний дараах `reviews.txt` 10
Зураг 4.16. Бодлого 4: 9 сэтгэгдлийн дахин шинжилгээ ба `results.txt` 10
Зураг 4.17. Бодлого 5: нэг сэтгэгдэл устгасны дараах `reviews.txt` 10
Зураг 4.18. Бодлого 6: `results.txt`-ээс N = 8 тэмдэгт унших 10
Зураг 4.19. Лаб 3.3-ын жишээ программын гаралт 11
Зураг 4.20. Өөрчилсөн программын гаралт (`lab3_3_1.py`) 11

ХҮСНЭГТИЙН ЖАГСААЛТ

Хүснэгт 1.1. `open()` функцийн лабораторийн ажилд хэрэглэсэн горимууд 2
Хүснэгт 2.1. Хэрэглэгдсэн программ хангамж, файлууд 4
Хүснэгт 5.1. Лаб 3.1-ийн бодлогуудын хүлээгдэж буй ба бодит үр дүн 12
Хүснэгт 5.2. Мэдрэмжийн шинжилгээний хүлээгдэж буй ба программын ангилал 14
Хүснэгт 5.3. Лаб 3.3-ын даалгаруудын хэрэгжүүлэлт 16

1. АЖЛЫН ЗОРИЛГО БА ОНОЛЫН ОЙЛГОЛТ

1.1. Ажлын зорилго

Лабораторийн ажлын зорилго нь Python хэл дээр нийлмэл өгөгдлийн төрлүүд (`list`, `tuple`, `set`, `dictionary`)-ийг бодлогын шинж чанарт тохируулан сонгож ашиглах, текст файлыг унших, бичих, нэмэх горимуудаар боловсруулах, класс болон удамшлыг ашиглан программын бүтэц зохион байгуулах, дараалал ба стекийн зарчмыг `deque` дээр хэрэгжүүлж туршихад оршсон [1]–[3]. Ажлыг Лаб 3.1, 3.2, 3.3 гэсэн гурван хэсэгт хуваан гүйцэтгэсэн бөгөөд хэсэг тус бүрд бие даан бодох бодлогуудыг шийдэж, программын гаралтыг дэлгэцийн зургаар баримтжуулсан. Тайланг [4]-т заасан загварын дагуу бэлтгэсэн.

1.2. List

`list` нь элементүүдийг оруулсан дарааллаар нь хадгалдаг, элементэд 0-ээс эхэлсэн индексээр хандаж болдог, үүсгэсний дараа агуулгыг нь өөрчилж болдог өгөгдлийн төрөл юм [5]. Элемент нэмэхэд `append()`, `extend()`, эрэмбэлэхэд `sort()`, устгахад `remove()`, `clear()` методыг хэрэглэдэг. `sort()` нь жагсаалтыг байрандаа өөрчилдөг бөгөөд `reverse=True` параметрээр буурах дарааллаар эрэмбэлнэ. Санамсаргүй бүхэл тоог `random.randint(a, b)` функцээр гаргах бөгөөд энэ функц a ба b хоёр захын утгыг оролцуулдаг [6].

1.3. Tuple

`tuple` нь `list`-тэй адил элементүүдийг дарааллаар нь хадгалдаг боловч үүсгэсний дараа элементийг нь өөрчлөх, нэмэх, хасах боломжгүй өгөгдлийн төрөл юм [5]. Иймээс (жимсний нэр, үнэ) гэх мэт тогтмол бүтэцтэй бичлэгийг хадгалахад тохиромжтой. Элементэд `list`-тэй адил индексээр хандана.

1.4. Set

`set` нь давхардаагүй элементүүдийн эрэмбэгүй цуглуулга юм [5]. Ижил утгыг дахин нэмэхэд олонлогт нэг л удаа үлддэг тул давхардал арилгах, өвөрмөц утгуудыг ялгахад ашиглана. Элемент нэмэхэд `add()` методыг хэрэглэнэ. Олонлог эрэмбэгүй тул элементүүдийн хэвлэгдэх дараалал тогтмол байх албагүй.

1.5. Dictionary

`dictionary` нь түлхүүр–утга (key–value) хосыг хадгалдаг бүтэц бөгөөд утгад түлхүүрээр нь шууд хандана [5]. Түлхүүр нь давхардахгүй, өөрчлөгддөггүй төрлийн (жишээ нь

тэмдэгт мөр) байх ёстой. `түлхүүр in dictionary` илэрхийлэл нь түлхүүр байгаа эсэхийг шалгадаг бол байхгүй түлхүүрт `dictionary[түлхүүр]` хэлбэрээр хандвал `KeyError` гарна; `get()` метод нь ийм тохиолдолд алдаа гаргалгүй `None` буцаадаг.

1.6. Файлтай ажиллах ба мэдрэмжийн шинжилгээ

Python-д файлыг `open(файлын_нэр, горим, encoding=...)` функцээр нээнэ [7], [8]. Горим нь файлд хийх үйлдлийг тодорхойлох бөгөөд лабораторийн ажилд хэрэглэсэн горимуудыг Хүснэгт 1.1-д үзүүлэв. `with` блок нь ажил дууссаны дараа файлыг автоматаар хаадаг тул `close()` дуудах шаардлагагүй болгодог [7]. `write()` метод нь мөр таслалыг өөрөө нэмдэггүй тул шинэ мөрөнд бичихийн тулд `'\n'` тэмдэгтийг тодорхой бичих хэрэгтэй. `read(n)` нь текст горимд эхний n тэмдэгтийг, `readlines()` нь бүх мөрийг жагсаалт болгон уншдаг. Кирилл үсэг агуулсан файлыг зөв унших, бичихийн тулд кодчилолыг `encoding="utf-8"` гэж заана; заагаагүй бол анхдагч кодчилол нь үйлдлийн системээс хамаарна [8].

Хүснэгт 1.1. `open()` функцийн лабораторийн ажилд хэрэглэсэн горимууд

Горим	Утга	Файл байхгүй үед	Байгаа агуулга
`'r'`	унших	`FileNotFoundError`	хадгалагдана
`'w'`	бичих	үүсгэнэ	устгагдана (файл тэглэгдэнэ)
`'a'`	төгсгөлд нэмэх	үүсгэнэ	хадгалагдаж, төгсгөлд нэмэгдэнэ
`'r+'`	унших ба бичих	`FileNotFoundError`	хадгалагдана

Мэдрэмжийн шинжилгээ (sentiment analysis) нь текстэд илэрсэн сэтгэл хөдлөлийг эерэг, сөрөг, төвийг сахисан гэж ангилдаг эх хэл боловсруулалтын бодлого юм [9]. Лабораторийн ажилд лексикон, өөрөөр хэлбэл урьдчилан бэлтгэсэн эерэг ба сөрөг үгсийн жагсаалт дээр суурилсан энгийн аргыг хэрэглэсэн. Жишээ программд эерэг үг нэг ч тохиолдвол «Positive», үгүй бөгөөд сөрөг үг тохиолдвол «Negative», аль нь ч байхгүй бол «Neutral» гэж ангилсан бол бие даалтад сэтгэгдэл дэх эерэг үгийн тоог p, сөрөг үгийн тоог q гэж тоолж, оноо S-ийг дараах томъёогоор тодорхойлсон:

$$S = p - q \tag{1.1}$$

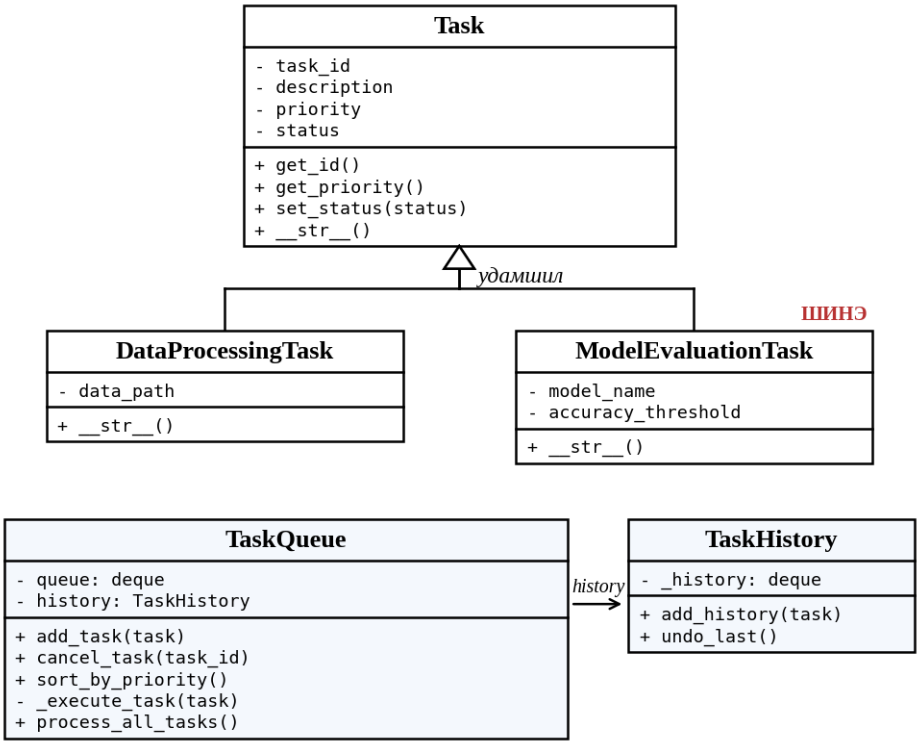
Энд S > 0 бол «Positive», S < 0 бол «Negative», S = 0 бол «Neutral» гэж ангилна. Энэ арга нь үгүйсгэлт (жишээ нь "not happy")-ийг ялгадаггүй бол VADER зэрэг дүрэмт системүүд үгүйсгэлтийг тусгай дүрмээр боловсруулдаг [10]. Ангиллын чанарыг зөв ангилагдсан сэтгэгдлийн хувиар үнэлнэ:

$$A = \frac{n_c}{n} \times 100\% \tag{1.2}$$

энд n_c нь зөв ангилагдсан сэтгэгдлийн тоо, n нь нийт сэтгэгдлийн тоо юм.

1.7. Класс ба удамшил

Класс нь өгөгдөл (шинж чанар) ба түүнийг боловсруулах функц (метод)-ийг нэгтгэсэн загвар юм [11]. `__init__` метод нь объектыг үүсгэх үед шинж чанарыг тохируулдаг, `__str__` метод нь объектыг тэмдэгт мөр болгон хэвлэх хэлбэрийг тодорхойлдог. Удамшлын үед хүүхэд анги (`class Child(Parent)`) эцэг ангийнхаа шинж чанар, методыг өвлөж авах бөгөөд `super()` функцээр эцэг ангийн методыг дууддаг; `isinstance(объект, анги)` функц нь объект тухайн ангийн (эсвэл түүнээс удамшсан ангийн) төлөөлөгч эсэхийг шалгана [11]. Алдааг `try–except` блокоор барьж, `raise` командаар зориуд үүсгэж болно [12]. Лаб 3.3-ын программын ангиудын бүтцийг Зураг 1.1-д үзүүлэв.



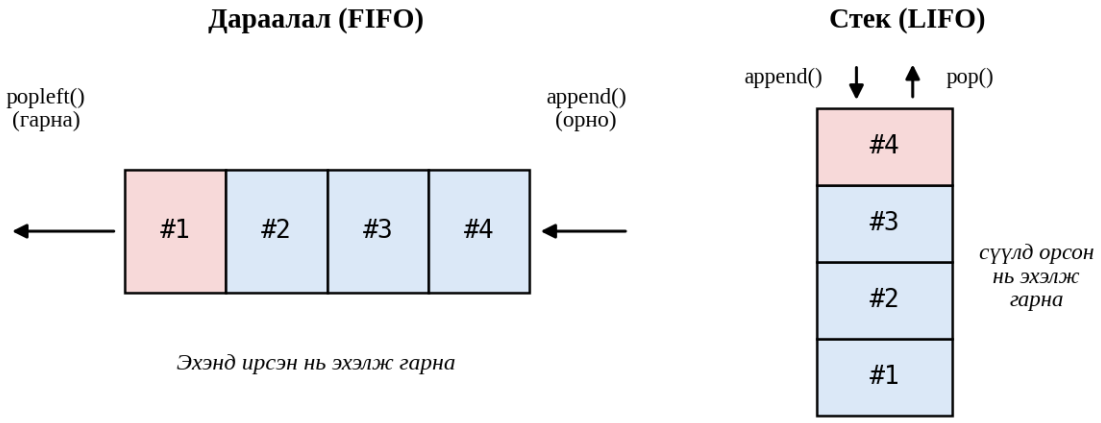
TaskQueue ба TaskHistory нь Task объектуудыг deque-д хадгалдаг

Зураг 1.1. Лаб 3.3-ын программын ангиудын бүтэц (UML диаграм)

1.8. Дараалал ба стек

Дараалал (queue) нь эхэнд орсон элемент эхэлж гардаг FIFO (First In, First Out), стек (stack) нь сүүлд орсон элемент эхэлж гардаг LIFO (Last In, First Out) зарчмаар ажилладаг өгөгдлийн бүтэц юм (Зураг 1.2). `collections.deque` нь хоёр үзүүрээс элемент нэмэх, авах үйлдлийг ойролцоогоор ижил, тогтмол хугацаанд гүйцэтгэдэг тул дараалал (`append()`, `popleft()`) ба стек (`append()`, `pop()`) хоёуланг хэрэгжүүлэхэд тохиромжтой [13]. Даалгаврыг тэргүүлэх зэргээр эрэмбэлэхэд `sorted(жагсаалт,`

`key=...)` функц хэрэглэх бөгөөд энэ эрэмбэлэлт нь тогтвортой (stable) буюу тэнцүү түлхүүртэй элементүүдийн анхны дараалыг хадгалдаг [14].



Зураг 1.2. Дараалал (FIFO) ба стек (LIFO)-ийн ажиллагааны схем

2. ХЭРЭГЛЭГДЭХҮҮН

Лабораторийн ажлыг гүйцэтгэхэд ашигласан программ хангамж, файлуудыг Хүснэгт 2.1-д жагсаав.

Хүснэгт 2.1. Хэрэглэгдсэн программ хангамж, файлууд

№	Хэрэгсэл	Тодорхойлолт	Зориулалт
1	Python	3.14.4 (GCC 15.2.0), Linux	бүх программыг бичих, ажиллуулах
2	IDLE	3.14.4	код бичих засварлагч (Editor), гаралт харах Shell
3	Стандарт сан	`random`, `collections` (`deque`)	санамсаргүй тоо (Лаб 3.1), дараалал ба стек (Лаб 3.3)
4	Текст файлууд	`myfile.txt`, `reviews.txt`, `results.txt`	файлын горимын жишээ, сэтгэгдлийн өгөгдөл, шинжилгээний үр дүн
5	Компьютер	Linux үйлдлийн систем	программ бичих, ажиллуулах, дэлгэцийн зураг авах

3. ХЭМЖИЛТ, ТУРШИЛТ ХИЙСЭН ДАРААЛАЛ

Программ бүрийг IDLE-ийн засварлагчид бичиж `lab3_1_1.py`, `lab3_1_2.py` гэх мэт нэрээр ажлын хавтсанд хадгалан, Run → Run Module (F5)-ээр ажиллуулж, гаралтыг Shell цонхноос дэлгэцийн зургаар авсан. Хэсэг тус бүрийн дараалыг доор үзүүлэв.

3.1. Лаб 3.1: нийлмэл өгөгдлийн төрлүүд

1) Жишээ 1 (`sum_list` функц) болон Жишээ 2 (номын сангийн программ)-ын кодыг ажиллуулж гаралтыг хадгалсан. Жишээ 2-т номын 3–4 нэрийг монгол номын нэрээр сольж дахин ажиллуулсан.

2) Бодлого 1: 8 элементтэй бүхэл тоон tuple үүсгэж, индекс бүрийг `for` давталтаар шалгаад индекс сондгой (`i % 2 == 1`) бөгөөд элемент сөрөг (`< 0`) бол `product` хувьсагчид үржүүлсэн; ийм элемент олдсон эсэхийг `found` хувьсагчаар тэмдэглэсэн.

3) Бодлого 2: бүхэл тоон жагсаалтын элемент бүрийг тэгш бол `B`, сондгой бол `C` жагсаалтад `append()`-ээр нэмж, `B.sort(reverse=True)`, `C.sort()` методоор эрэмбэлж хэвлэсэн.

4) Бодлого 3: Монгол нэр томьёог түлхүүр, англи орчуулгыг утга болгосон 12 элементтэй `dictionary` үүсгэж, `input()`-аар авсан үгийг `lower()`-оор жижиг үсэг болгон `in` үйлдлээр шалгаж, олдвол орчуулгыг хэвлэсэн.

5) Бодлого 4: `random` модулийг импортлож, `for` давталтаар `random.randint(10, 100)` утгыг 10 удаа жагсаалтад нэмж хэвлэсэн.

6) Бодлого 5: жимсний нэрсийн жагсаалт (`list`), жимс бүрийн өнгийн `dictionary`, өвөрмөц өнгөний `set`, (нэр, үнэ) `tuple`-үүдийн жагсаалт үүсгэж, `find_fruits_by_color(color)`, `find_fruits_by_price(price)`, `show_summary()` функцийг бичиж, функц бүрд docstring нэмсэн. Өнгө, үнийг гарнаас уншиж хайлт хийсэн.

3.2. Лаб 3.2: файлтай ажиллах

1) Жишээ 1: `myfile.txt` файлыг `'w'` горимоор нээж гурван мөр бичсэн, дараа нь `'a'` горимоор «Today» нэмж `'r'` горимоор уншсан; эцэст нь `'w'` горимоор «Tomorrow» бичиж дахин уншсан.

2) Жишээ 2: `'a'` горимд нэмэхийн өмнө шинэ мөрийн тэмдэг `'\n'` бичсэн болон бичээгүй тохиолдлыг харьцуулсан.

3) Жишээ 3: `with` хэллэг ашиглан `'w'`, `'a'`, `'r+'` горимоор бичиж, уншсан.

4) Бодлого 1: 6 сэтгэгдлийг мөр тус бүрт нэгээр `reviews.txt` файлд `'w'` горимоор бичсэн. Дараа нь мэдрэмжийн шинжилгээний жишээ программыг ажиллуулж, үр дүнг `results.txt`-д хадгалуулсан.

5) Бодлого 2: `reviews.txt`-ийн мөр бүрийг `readlines()`-ээр уншиж, `lower()`-оор жижиг үсэг болгон эерэг, сөрөг үгийн тоог тоолж, ангилалыг `results.txt`-д бичсэн.

6) Бодлого 3: `'a'` горимоор 3 шинэ сэтгэгдэл нэмсэн. Бодлого 4: шинжилгээг 9 сэтгэгдэлд дахин хийсэн.
7) Бодлого 5: файлын бүх мөрийг уншиж, устгах сэтгэгдлийг хасаад `'w'` горимоор дахин бичсэн.
8) Бодлого 6: `input()`-аар N тоог уншиж, `results.txt`-ээс `read(N)`-ээр эхний N тэмдэгтийг уншиж хэвлэсэн (N = 8).

3.3. Лаб 3.3: класс, удамшил, дараалал, стек

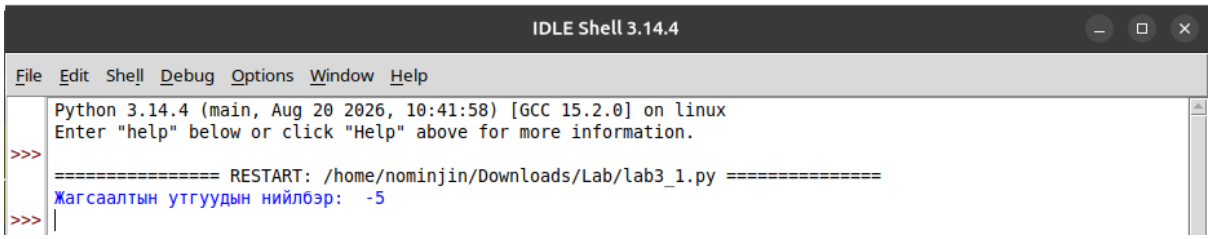
1) Жишээ программыг ажиллуулж, даалгаврууд хэрхэн боловсруулж буйг ажигласан.
2) Даалгавар 2–7-г хэрэгжүүлсэн: `Task`-аас удамшсан `ModelEvaluationTask` анги нэмсэн; `_execute_task()`-д `isinstance()`-ээр даалгаврын төрөл тус бүрд өөр боловсруулалт хийлгэсэн; `cancel_task()`, `sort_by_priority()` методыг бичсэн; `try–except`, `raise ValueError`, `raise IndexError`-ээр алдааг зохицуулсан; анги, метод бүрд docstring нэмсэн.
3) Туршилтад T101 (`Task`, priority 3), T102 (`DataProcessingTask`, priority 1), T103 (`ModelEvaluationTask`, priority 2), T104 (замгүй `DataProcessingTask`, priority 1) даалгаврыг үүсгэн дараалалд нэмсэн; T101-ийг цуцалсан; `sort_by_priority()`-ээр эрэмбэлж, `process_all_tasks()`-ээр боловсруулсан; `undo_last()`-ээр түүхийн стекээс сүүлийн даалгаврыг буцаасан.

4. ХЭМЖИЛТ, ТУРШИЛТЫН ҮР ДҮНГҮҮД

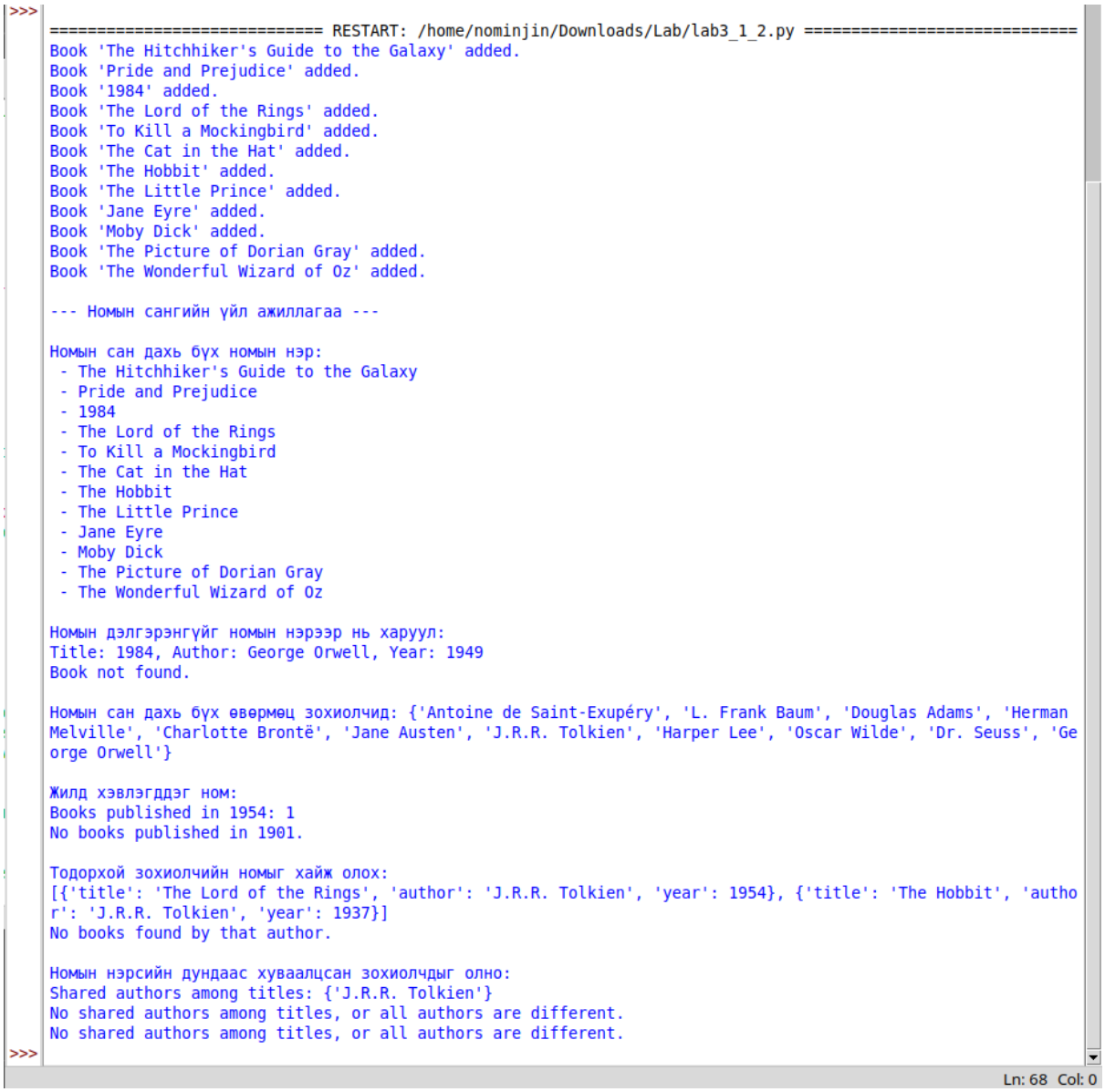
Энэ бүлэгт программуудын гаралтыг боловсруулаагүй хэлбэрээр дэлгэцийн зургаар үзүүлэв. Үр дүнгийн шинжилгээг 5-р бүлэгт хийсэн бөгөөд программын кодыг хавсралтад байрлуулав.

4.1. Лаб 3.1-ийн үр дүн

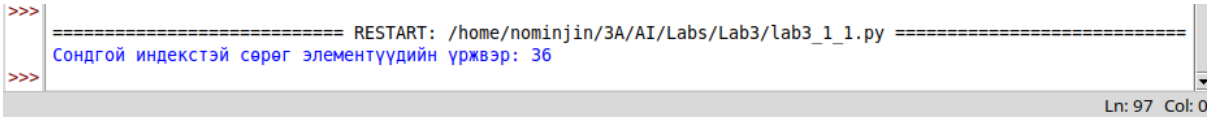
Жишээ 1, 2-ын гаралтыг Зураг 4.1–4.3-д, бие даалтын Бодлого 1–5-ын гаралтыг Зураг 4.4–4.8-д үзүүлэв. Код нь Хавсралт А-д байна.



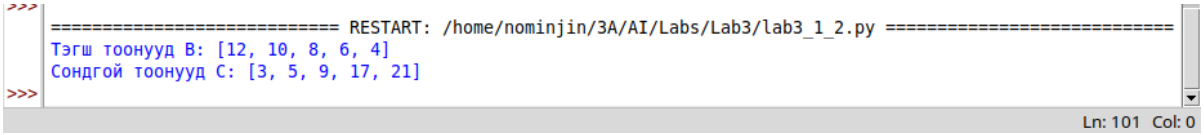
Зураг 4.1. Жишээ 1: `sum_list` функцийн гаралт



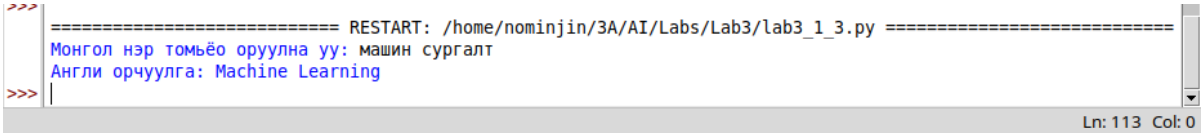
Зураг 4.2. Жишээ 2: номын сангийн программын гаралт (анхны нэрс)



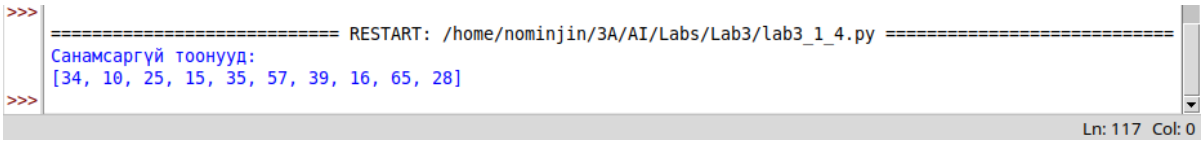
Зураг 4.4. Бодлого 1: сондгой индекстэй сөрөг элементүүдийн үржвэр



Зураг 4.5. Бодлого 2: тэгш ба сондгой тоонуудын жагсаалт



Зураг 4.6. Бодлого 3: Монгол–Англи нэр томьёоны толь

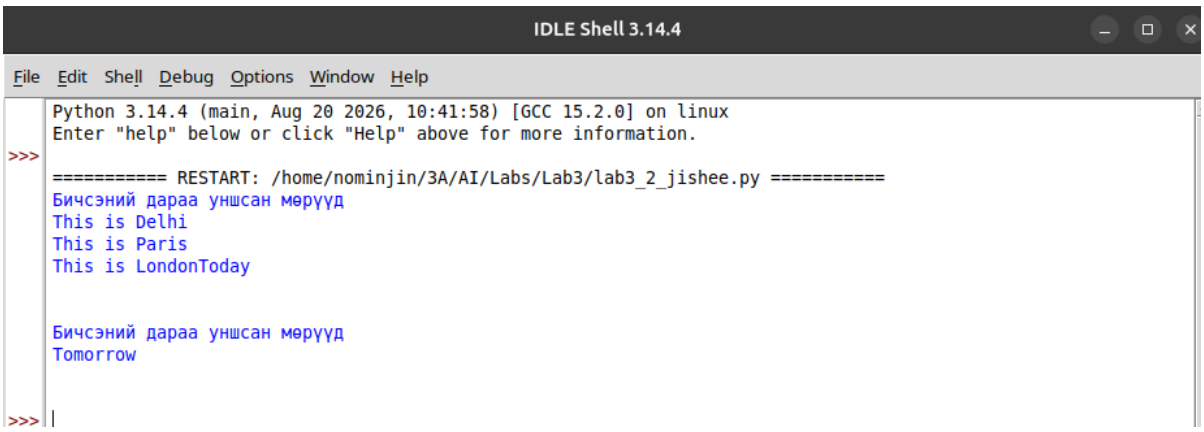


Зураг 4.7. Бодлого 4: санамсаргүй бүхэл тоонуудын жагсаалт

Зураг 4.8. Бодлого 5: жимсний мэдээлэл боловсруулах программ

4.2. Лаб 3.2-ийн үр дүн

Файлын горимын Жишээ 1–3-ын гаралтыг Зураг 4.9–4.11-д, мэдрэмжийн шинжилгээний жишээ болон бие даалтын алхам бүрийн гаралтыг Зураг 4.12–4.18-д үзүүлэв. Файлын агуулгыг харуулсан зурагт `cat` командын хэлбэрээр харуулав. Код нь Хавсралт Б-д байна.



Зураг 4.9. Жишээ 1: `'w'`, `'a'`, `'r'` горимоор бичих, унших

```
========== RESTART: /home/nominjin/3A/AI/Labs/Lab3/lab3_2_jishee2.py ==========
Бичсэний дараа уншсан мөрүүд
This is Delhi
This is Paris
This is London
TodayTomorrow

>>>
```

Зураг 4.10. Жишээ 2: шинэ мөрийн тэмдэгтэй болон тэмдэггүй нэмэх

```
File  Edit  Shell  Debug  Options  Window  Help
Бичсэний дараа уншсан мөрүүд
Tomorrow


>>>
========== RESTART: /home/nominjin/3A/AI/Labs/Lab3/lab3_2_jishee.py ==========
Бичсэний дараа уншсан мөрүүд
This is Delhi
This is Paris
This is LondonToday


Бичсэний дараа уншсан мөрүүд
Tomorrow


>>>
========== RESTART: /home/nominjin/3A/AI/Labs/Lab3/lab3_2_jishee2.py ==========
Бичсэний дараа уншсан мөрүүд
This is Delhi
This is Paris
This is London
TodayTomorrow
```

Зураг 4.11. Жишээ 3: `with` хэллэг ашиглан бичих, унших

```
>>>
========== RESTART: /home/nominjin/3A/AI/Labs/Lab3/lab3_2_1.py ==========
reviews.txt файл үүслээ.
>>>
```

Ln: 54 Col: 0

```
$ cat reviews.txt
I love this product, it's amazing!
Worst experience ever, totally disappointed.
The service was okay, nothing special.
Absolutely fantastic! Highly recommend.
Not worth the price, I regret buying it.
Decent quality, but could be better.
```

Зураг 4.12. Бодлого 1: `reviews.txt` файл үүсгэсэн байдал

```
$ python3 lab3_2_jishee_4.py
Sentiment analysis complete! Check 'results.txt' for results.
$ cat results.txt
I love this product, it's amazing! -> Sentiment: Positive
Worst experience ever, totally disappointed. -> Sentiment: Negative
The service was okay, nothing special. -> Sentiment: Neutral
Absolutely fantastic! Highly recommend. -> Sentiment: Positive
Not worth the price, I regret buying it. -> Sentiment: Negative
Decent quality, but could be better. -> Sentiment: Neutral
```

Зураг 4.13. Мэдрэмжийн шинжилгээний жишээ программын гаралт ба `results.txt`

```
>>>
========== RESTART: /home/nominjin/3A/AI/Labs/Lab3/lab3_2_2.py ==========
Sentiment analysis дууслаа.
>>>
```

Ln: 19 Col: 0

```
$ cat results.txt
I love this product, it's amazing! -> Positive
Worst experience ever, totally disappointed. -> Negative
The service was okay, nothing special. -> Neutral
Absolutely fantastic! Highly recommend. -> Positive
Not worth the price, I regret buying it. -> Negative
Decent quality, but could be better. -> Positive
```

Зураг 4.14. Бодлого 2: мэдрэмжийн шинжилгээний гаралт ба `results.txt`

```
>>>
========== RESTART: /home/nominjin/3A/AI/Labs/Lab3/lab3_2_3.py ==========
Шинэ review-үүд нэмэгдлээ.
>>>
```

Ln: 60 Col: 0

```
$ cat reviews.txt
I love this product, it's amazing!
Worst experience ever, totally disappointed.
The service was okay, nothing special.
Absolutely fantastic! Highly recommend.
Not worth the price, I regret buying it.
Decent quality, but could be better.
The product is amazing.
I am not happy with this service.
Good quality and fast delivery.
```

Зураг 4.15. Бодлого 3: `'a'` горимоор нэмсний дараах `reviews.txt`

```
$ python3 lab3_2_4.py
Шинэ review-үүдийг дахин шинжилж дууслаа.
```

```
$ cat results.txt
I love this product, it's amazing! -> Positive
Worst experience ever, totally disappointed. -> Negative
The service was okay, nothing special. -> Neutral
Absolutely fantastic! Highly recommend. -> Positive
Not worth the price, I regret buying it. -> Negative
Decent quality, but could be better. -> Positive
The product is amazing. -> Positive
I am not happy with this service. -> Positive
Good quality and fast delivery. -> Positive
```

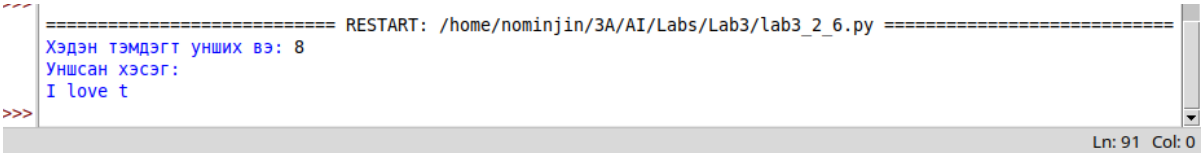
Зураг 4.16. Бодлого 4: 9 сэтгэгдлийн дахин шинжилгээ ба `results.txt`

```
>>>
========== RESTART: /home/nominjin/3A/AI/Labs/Lab3/lab3_2_5.py ==========
Сонгосон review устгагдлаа.
>>>
```

Ln: 86 Col: 0

```
$ cat reviews.txt
I love this product, it's amazing!
Worst experience ever, totally disappointed.
Absolutely fantastic! Highly recommend.
Not worth the price, I regret buying it.
Decent quality, but could be better.
The product is amazing.
I am not happy with this service.
Good quality and fast delivery.
```

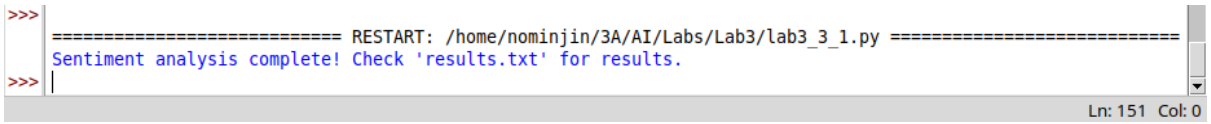
Зураг 4.17. Бодлого 5: нэг сэтгэгдэл устгасны дараах `reviews.txt`



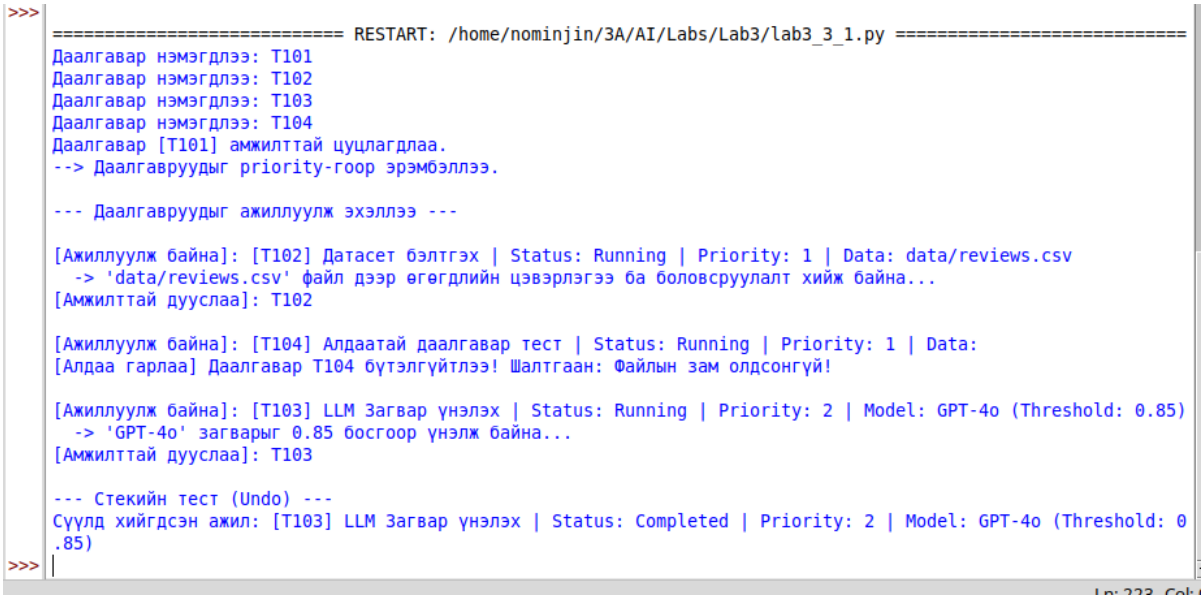
Зураг 4.18. Бодлого 6: `results.txt`-ээс N = 8 тэмдэгт унших

4.3. Лаб 3.3-ын үр дүн

Жишээ программын гаралтыг Зураг 4.19-д, өөрчилсөн программын гаралтыг Зураг 4.20-д үзүүлэв. Код нь Хавсралт В-д байна.



Зураг 4.19. Лаб 3.3-ын жишээ программын гаралт



Зураг 4.20. Өөрчилсөн программын гаралт (`lab3_3_1.py`)

5. ХЭЛЭЛЦҮҮЛЭГ

Энэ бүлэгт 4-р бүлэгт үзүүлсэн гаралтыг хүлээгдэж буй үр дүн болон онолын ойлголттой харьцуулан тайлбарлаж, ажиглагдсан дутагдлыг шинжилсэн.

5.1. Лаб 3.1-ийн үр дүнгийн шинжилгээ

Бодлого бүрийн оролт, хүлээгдэж буй ба программын гаралтыг Хүснэгт 5.1-д харьцуулав.

Хүснэгт 5.1. Лаб 3.1-ийн бодлогуудын хүлээгдэж буй ба бодит үр дүн

Бодлого	Оролт	Хүлээгдэж буй үр дүн	Программын гаралт	Харьцуулалт
1	`(5, -2, 7, -3, 4, -6, 8, -1)`	(−2)·(−3)·(−6)·(−1) = 36	36	тохирсон
2	бүхэл тоон жагсаалт	B — тэгш, буурах; C — сондгой, өсөх	B = `[12, 10, 8, 6, 4]`; C = `[3, 5, 9, 17, 21]`	тохирсон
3	«машин сургалт»	Machine Learning	Machine Learning	тохирсон
4	`random.randint(10, 100)` × 10	10 тоо, бүгд 10–100 завсарт	`[34, 10, 25, 15, 35, 57, 39, 16, 65, 28]`	тохирсон
5	өнгө «шар», үнэ 7000; өнгө «цэнхэр», үнэ 4500	`['Banana', 'Mango']`, `['Orange']`; `[]`, «олдсонгүй»	хүлээгдэж буйтай ижил (Зураг 4.8)	тохирсон

Жишээ 1, 2. Жишээ 2-т list (`library_books`), set (`unique_authors`), dictionary (`book_details`), tuple (`book_counts_per_year`) гэсэн дөрвөн төрлийг нэг программд хослуулсан бөгөөд төрөл бүр өөр зорилгоор хэрэглэгддэг [1]: жагсаалт нь ном бүрийн мэдээллийг дарааллаар нь хадгалж, олонлог нь олон номтой зохиогчийг ч нэг л удаа бүртгэж, толь бичиг нь номыг нэрээр нь шууд хайх боломж олгож, tuple нь (он, номын тоо) хосыг өөрчлөгдөхгүй байдлаар хадгалдаг. Номын 3–4 нэрийг монгол нэрээр сольж ажиллуулсан туршилт (Зураг 4.3) нь кирилл тэмдэгт мөр толь бичгийн түлхүүр, олонлогийн элемент болоход программ өөрчлөлтгүй ажиллаж байгаа эсэхийг шалгах зорилготой байсан. Жишээ 1-д `sum_list` функц жагсаалтын элементүүдийг давталтаар нэмж нийлбэрийг буцаадаг (Зураг 4.1).

Бодлого 1. Сондгой индекс 1, 3, 5, 7 дээрх утгууд −2, −3, −6, −1 бөгөөд бүгд сөрөг тул дөрвөн сөрөг тооны үржвэр эерэг гарч 36 болсон (Зураг 4.4). Индекс 0-ээс эхэлдэг тул `i % 2 == 1` нөхцөл нь 2-р, 4-р, 6-р, 8-р байрлалын элементийг сонгодог. Гэвч туршилтын tuple-д бүх сөрөг тоо сондгой индекс дээр, бүх эерэг тоо тэгш индекс дээр байрласан тул нөхцөл тус бүрийг тусад нь шалгаж чадаагүй: аль нэг нөхцөл дутсан ч (жишээ нь зөвхөн `numbers[i] < 0`) үр дүн 36 хэвээр гарах байсан. Хоёр нөхцлийг бүрэн шалгахын тулд сондгой индекс дээр эерэг, тэгш индекс дээр сөрөг утга агуулсан tuple-ээр давтан

туршихыг зөвлөнө. `found` хувьсагч нь тохирох элемент огт байхгүй үед `product`-ын анхны утга 1-ийг буруугаар үр дүн болгон хэвлэхээс сэргийлдэг.

Бодлого 2. Гаралтад B = `[12, 10, 8, 6, 4]` буурах, C = `[3, 5, 9, 17, 21]` өсөх дарааллаар байгаа нь `B.sort(reverse=True)` ба `C.sort()` методын үр дүн юм (Зураг 4.5). `sort()` нь жагсаалтыг байрандаа өөрчилдөг тул эрэмбэлсний дараа `append()`-ээр нэмсэн анхны дараалал алдагдана [5]. Тэгш тоо (12, 10, 8, 6, 4) бүгд B-д, сондгой тоо (3, 5, 9, 17, 21) бүгд C-д орсон нь тэгш, сондгойг ялгах нөхцөл зөв ажилласныг харуулж байна.

Бодлого 3. 12 нэр томьёотой толь бичгээс «машин сургалт»-ын англи орчуулга Machine Learning гарсан (Зураг 4.6). `lower()` нь оролтыг жижиг үсэг болгож, том үсгээр бичсэн үгийг ч таних боломж олгодог бөгөөд кирилл үсэгт ч зөв ажилладаг. Харин `strip()` ашиглаагүй тул үгийн урд, ард хоосон зай орсон оролт тольд олдохгүй. Мөн монгол хэлний нэр үг тийн ялгалын нөхцөлөөр хувирдаг (сургалт → сургалтын) тул хувирсан хэлбэрийг таньдаггүй. Туршилтад зөвхөн байгаа үг оруулсан тул «Тольд байхгүй байна.» салаа ажиллаагүй; хоёр салааг хоёуланг нь туршихыг зөвлөнө.

Бодлого 4. `random.randint(10, 100)` нь 10 ба 100-ыг оролцуулан бүхэл тоо буцаадаг [6]. Гарсан 10 тоо бүгд 10–100 завсарт байгаа бөгөөд хамгийн бага нь 10 байсан нь доод захын утга оролцдог гэсэн баримт бичгийн тодорхойлолттой нийцэж байна, гэхдээ нэг удаагийн ажиллуулалт үүнийг нотлохгүй (Зураг 4.7). Санамсаргүй тоо үүсгэгчийг `seed`-гүй ажиллуулсан тул ажиллуулах бүрд гаралт өөр байх бөгөөд үр дүнг яг ижлээр давтан гаргахын тулд `random.seed()` ашиглах хэрэгтэй.

Бодлого 5. Жимсний программ дөрвөн төрлийг тус бүр өөр зорилгоор ашигласан (Зураг 4.8): нэрсийн жагсаалт (list), нэр → өнгө харгалзуулсан толь бичиг (dictionary), өвөрмөц өнгөний олонлог (set), (нэр, үнэ) хосын tuple жагсаалт. 4 жимсэнд 3 өвөрмөц өнгө (улаан, улбар шар, шар) гарсан нь «шар» өнгө Banana, Mango хоёрт давхардсныг олонлог автоматаар арилгасныг харуулж байна. Олонлог эрэмбэгүй тул `sorted()`-ээр эрэмбэлж харуулсан. Өнгө «шар»-аар хайхад Banana, Mango гэсэн хоёр нэр, үнэ 7000-аар хайхад Orange гарсан. Байхгүй өнгө (цэнхэр), үнэ (4500)-д функц алдаа өгөхгүй хоосон жагсаалт буцааж, дуудсан тал `if result:` шалгалтаар «олдсонгүй» гэж мэдэгдсэн. Функц бүрт docstring бичсэн нь кодын баримтжуулалтын шаардлагыг хангасан.

5.2. Лаб 3.2-ийн үр дүнгийн шинжилгээ

Файлын горимууд (Жишээ 1–3). Жишээ 1-д (Зураг 4.9) `'w'` горимоор бичсэн гурав дахь мөр «This is London» нь `'\n'`-гүй төгссөн тул `'a'` горимоор нэмсэн «Today» түүний үргэлжлэл болж «This is LondonToday» хэлбэрээр уншигдсан. Дараа нь `'w'` горимоор «Tomorrow» бичихэд өмнөх бүх агуулга устаж зөвхөн «Tomorrow» үлдсэн нь `'w'` горим файлыг нээх агшинд тэглэдгийг харуулж байна [7], [8]. Жишээ 2-т (Зураг 4.10) нэмэхийн өмнө `'\n'` бичсэн тул «Today» шинэ мөрөнд орсон боловч «Today», «Tomorrow»-г дараалан бичихдээ `'\n'` оруулаагүй тул «TodayTomorrow» болж нийлсэн; иймд `write()` шинэ мөрийг өөрөө нэмдэггүй нь батлагдсан. Мөн Жишээ 2-т `'a'` горимоор нээсэн файлыг `close()` хийлгүйгээр дахин нээсэн боловч өгөгдөл зөв уншигдсан нь CPython хуучин файлын объектыг устгахдаа буферийг файлд бичдэгтэй холбоотой; энэ нь Python-ы хэрэгжүүлэлтээс хамаардаг тул найдваргүй бөгөөд Жишээ 3-т (Зураг 4.11) ашигласан `with` блок нь файлыг ямагт хаадаг тул илүү зөв арга юм [7]. Жишээ 3-ын гаралтад «Hello» эхэнд, «Today» төгсгөлд байгаа нь `'w'`, дараа нь `'a'` горимоор бичсэн дарааллыг харуулж байна.

Мэдрэмжийн шинжилгээний үнэлгээ. Жишээ программын (Зураг 4.13) болон бие даалтын (Зураг 4.14, Зураг 4.16) шинжилгээг сэтгэгдэл бүрийг гараар ангилсан хүлээгдэж буй ангилалтай Хүснэгт 5.2-д харьцуулав.

Хүснэгт 5.2. Мэдрэмжийн шинжилгээний хүлээгдэж буй ба программын ангилал

№	Сэтгэгдэл	Гараар ангилсан	Жишээ программ	Бие даалтын программ	Бие даалт зөв эсэх
1	I love this product, it's amazing!	Positive	Positive	Positive	тийм
2	Worst experience ever, totally disappointed.	Negative	Negative	Negative	тийм
3	The service was okay, nothing special.	Neutral	Neutral	Neutral	тийм
4	Absolutely fantastic! Highly recommend.	Positive	Positive	Positive	тийм
5	Not worth the price, I regret buying it.	Negative	Negative	Negative	тийм
6	Decent quality, but could be better.	Neutral	Neutral	Positive	**үгүй**
7	The product is amazing.	Positive	—	Positive	тийм
8	I am not happy with this service.	Negative	—	Positive	**үгүй**
9	Good quality and fast delivery.	Positive	—	Positive	тийм

«—»: жишээ программ зөвхөн анхны 6 сэтгэгдэлд ажилласан.

Бие даалтын программ 9 сэтгэгдлээс 7-г зөв ангилсан тул (1.2) томъёогоор A = 7 / 9 × 100 % = 77.8 % байна. Жишээ программ анхны 6 сэтгэгдлийг бүгдийг нь зөв ангилсан (6 / 6). Түүвэр 6–9 сэтгэгдэлтэй тул нэг сэтгэгдлийн алдаа үзүүлэлтийг 11–17 нэгжээр

өөрчилдөг; иймд эдгээр хувь нь аргын чанарыг бүрэн илэрхийлэхгүй, зөвхөн жишээн дээрх ажиглалт юм. Ангилалыг гараар тодорхойлсон нь хувь хүний үнэлгээ тул 6-р сэтгэгдлийг («Decent quality, but could be better.») зарим уншигч бага зэрэг эерэг гэж үзэж болно.

Алдаатай ангилал. 6-р сэтгэгдэл дээр хоёр программын үр дүн зөрсөн: жишээ программын эерэг жагсаалтад «decent» үг байхгүй тул «Neutral» гарсан бол бие даалтад «decent»-ийг эерэг жагсаалтад нэмсэн тул p = 1, q = 0, S = 1 болж «Positive» гарсан. Энэ нь лексикон дээр суурилсан арга үгсийн жагсаалтаас шууд хамаарахыг, «but could be better» гэсэн хоёрдмол утгыг үгсийн тоолоloор илэрхийлэх боломжгүйг харуулж байна. 8-р сэтгэгдэл («I am not happy with this service.») нь сөрөг агуулгатай боловч «happy» үг эерэг жагсаалтад байгаа тул p = 1, q = 0 болж «Positive» гэж буруу ангилагдсан. Программ `'not worth'` гэсэн нэг хэллэгийг сөрөг жагсаалтад тусад нь оруулсан боловч «not happy» гэх мэт бусад үгүйсгэлийг хамраагүй; хэллэгийг нэг нэгээр нь нэмэх нь өргөтгөх боломжгүй тул үгүйсгэх үг (`not`, `no`, `never`)-ийн дараах үгийн утгыг эсрэгээр болгох дүрэм хэрэгтэй бөгөөд VADER иймарга хэрэглэдэг [10]. Мөн `word in text` нь тэмдэгт мөрийн хэсэг тохирлыг шалгадаг тул үгийн нэг хэсэг болсон тохиолдолд ч тоологдох эрсдэлтэй. Ангиллын тоогоор: бие даалтын программ 6 сэтгэгдлийг «Positive», 2-ыг «Negative», 1-ийг «Neutral» гэсэн бол гараар ангилахад 4 эерэг, 3 сөрөг, 2 төвийг сахисан байв; иймд программ эерэг ангиллыг илүү, сөрөг ба төвийг сахисан ангиллыг дутуу тоолсон.

Файлыг шинэчлэх (Бодлого 3–5). Бодлого 3-т `'a'` горимоор нэмсэн гурван сэтгэгдэл `reviews.txt`-ийн төгсгөлд орж мөрийн тоо 6-аас 9 болсон бөгөөд өмнөх агуулга хадгалагдсан (Зураг 4.15). Бодлого 1-ийн код сэтгэгдэл бүрийн ард `'\n'` бичдэг тул файл мөр таслалаар төгссөн байсан бөгөөд шинэ сэтгэгдлүүд шинэ мөрөөс эхэлсэн; Жишээ 1-д ажигласан шиг мөр таслалгүй төгссөн файлд `'a'` горимоор бичвэл сүүлийн мөртэй нийлэх байсан. Бодлого 4-т `results.txt`-ийг `'w'` горимоор нээсэн тул хуучин 6 мөр устаж, 9 сэтгэгдлийн үр дүн шинээр бичигдсэн (Зураг 4.16). Бодлого 5-д текст файлаас нэг мөрийг шууд устгах функц байдаггүй тул бүх мөрийг `readlines()`-ээр уншиж, `strip()`-ээр мөр таслалыг хасаж харьцуулан устгах сэтгэгдлийг алгасаад `'w'` горимоор дахин бичсэн; `reviews.txt` 9 мөрөөс 8 мөр болсон (Зураг 4.17). Энэ арга нь `'w'` горим файлыг нээх агшинд тэглэдэг тул дахин бичих явцад программ зогсвол өгөгдөл алдагдах эрсдэлтэй. Мөн устгасны дараа шинжилгээг дахин ажиллуулаагүй тул `results.txt` нь

устгасан сэтгэгдлийн мөрийг агуулсан хэвээр байгаа бөгөөд `reviews.txt`-тэй зөрөх боломжтой.

Тэмдэгт унших (Бодлого 6). N = 8 гэж оруулахад `results.txt`-ийн эхний 8 тэмдэгт «I love t» хэвлэгдсэн (Зураг 4.18); хоосон зай мөн тэмдэгтэд тооцогдсон. Текст горимд `read(n)` байт биш тэмдэгт уншдаг [7]. Программ оролтыг шалгадаггүй: тоо биш утга оруулбал `int()` функц `ValueError` гаргаж программ зогсох бөгөөд сөрөг тоо оруулбал `read(-1)` нь файлыг бүхэлд нь уншина. Иймд оролтыг `try–except` блокоор шалгаж, N ≥ 0 эсэхийг нягтлах нь зөв.

5.3. Лаб 3.3-ын үр дүнгийн шинжилгээ

Даалгавар 1–7-г хэрэгжүүлсэн байдлыг Хүснэгт 5.3-д нэгтгэж, өөрчилсөн программын гаралтыг (Зураг 4.20) дараах байдлаар шинжилсэн.

Хүснэгт 5.3. Лаб 3.3-ын даалгаврууд хэрэгжүүлэлт

Даалгавар	Хэрэгжүүлэлт	Гүйцэтгэл
1. Жишээг ажиллуулах	Жишээ программыг ажиллуулж, даалгавар боловсруулагдах дарааллыг ажигласан (Зураг 4.19)	гүйцэтгэсэн
2. Шинэ subclass	`ModelEvaluationTask` — `model_name`, `accuracy_threshold` шинж чанартай, `Task`-аас удамшсан	гүйцэтгэсэн
3. `_execute_task`-г сайжруулах	`isinstance()`-ээр даалгаврын төрлийг ялгаж `DataProcessingTask` (замын шалгалт), `ModelEvaluationTask` (босгын шалгалт), бусад төрөлд өөр өөр боловсруулалт хийдэг	гүйцэтгэсэн (энгийн түвшинд)
4. Даалгавар цуцлах	`cancel_task(task_id)` — дарааланд хүлээгдэж буй даалгаврыг хасаж «Cancelled» төлөвтэй болгон түүхэд нэмдэг	хэсэгчлэн (эхэлсэн даалгаварт үйлчлэхгүй)
5. Тэргүүлэх зэрэг	`sort_by_priority()` — `sorted(..., key=get_priority)`	гүйцэтгэсэн
6. Алдааны зохицуулалт	`try–except`, `ValueError` (хоосон зам, босго), `IndexError` (хоосон стек); мессежэд даалгаврын ID, шалтгаан орсон	гүйцэтгэсэн
7. Баримтжуулалт	анги, метод бүрт docstring, гол хэсэгт тайлбар	гүйцэтгэсэн

Ангийн бүтэц ба удамшил. `DataProcessingTask`, `ModelEvaluationTask` нь `Task`-ийн `task_id`, `description`, `priority`, `status` шинж чанар болон методуудыг `super().__init__()`-ээр өвлөж, зөвхөн өөрийн шинж чанарыг (`data_path`; `model_name`, `accuracy_threshold`) нэмсэн (Зураг 1.1). `__str__()` метод нь эцэг ангийн мөрийг `super().__str__()`-ээр авч өөрийн мэдээллийг залгадаг тул гаралтад T102 «... | Data: data/reviews.csv», T103 «... | Model: GPT-4o (Threshold: 0.85)» хэлбэрээр тус тусын нэмэлт мэдээлэлтэй хэвлэгдсэн (Зураг 4.20).

Цуцлалт. T101 (`Task`, priority 3) нь `cancel_task("T101")` дуудагдахад дарааллаас хасагдаж «Cancelled» төлөвтэй болон түүхийн стекэд нэмэгдсэн тул боловсруулалтын логт байхгүй. Хэрэгжүүлэлт нь зөвхөн дараалалд хүлээгдэж буй даалгаварт үйлчилдэг: `process_all_tasks()` даалгаврыг `popleft()`-ээр дарааллаас авдаг тул эхэлсэн даалгаврыг ID-аар нь олж «цуцалсан» гэж тэмдэглэх боломжгүй бөгөөд лабораторийн зааврын 4-р даалгаврын хоёр дахь нөхцөл хэрэгжээгүй хэвээр байна [3].

Тэргүүлэх зэргээр эрэмбэлэх. Цуцлалтын дараа дараалал T102 (1), T103 (2), T104 (1) байсан бол `sort_by_priority()`-ийн дараа T102, T104, T103 болж, боловсруулалт яг энэ дарааллаар явагдсан. Тэргүүлэх зэрэг ижил (1) T102 ба T104 ирсэн дарааллаа хадгалсан нь `sorted()` тогтвортой эрэмбэлэлт гэдгийг харуулж байна [14]. Зэргийн тоо бага байх тусам өндөр зэрэглэлтэй гэж тооцсон. FIFO дарааллаар T103 нь T104-ээс өмнө гүйцэтгэгдэх байсан бол эрэмбэлсний дараа T104 түүнээс түрүүлсэн.

Алдааны зохицуулалт. T104 нь хоосон `data_path`-тай тул `_execute_task()` дотор `ValueError` үүсч, `except Exception` блокт баригдан төлөв «Failed» болж, даалгаврын ID болон шалтгаан агуулсан «Даалгавар T104 бүтэлгүйтлээ! Шалтгаан: Файлын зам олдсонгүй!» мессеж хэвлэгдсэн. Нэг даалгавар бүтэлгүйтсэн ч дараалал зогсоогүй: T104-ийн дараа T103 үргэлжлэн амжилттай гүйцэтгэгдсэн. Бүтэлгүйтсэн даалгавар түүхийн стекэд нэмэгдээгүй тул стекэд зөвхөн T101 (цуцлагдсан), T102, T103 байсан.

Стек. `undo_last()` дуудахад сүүлд түүхэд орсон T103 буцаагдсан нь LIFO зарчимтай нийцэв. Хоосон стекээс `pop()` хийвэл `IndexError` гардаг тул `undo_last()` нь эхлээд `if not self._history` гэж шалгаад «Стек хоосон байна!» мессеж хэвлэж `None` буцаадаг; гэвч энэ салаа туршилтад ажиллаагүй.

Туршилтын хамрах хүрээ ба сайжруулалт. Энэ туршилтад `ModelEvaluationTask`-ийн босгын (0–1) алдааны салаа, үндсэн `Task` төрлийн ерөнхий боловсруулалт (T101 цуцлагдсан тул), хоосон стекийн салаа гурвуулаа ажиллаагүй. Эдгээрийг шалгахын тулд босго нь 1.5 гэсэн даалгавар нэмэх, `undo_last()`-ийг дараалан 4 удаа дуудах тест хэрэгтэй. Мөн `heapq` модулийг импортолсон боловч ашиглаагүй байгаа тул хасах (эсвэл тэргүүлэх дарааллыг `heapq`-ээр хэрэгжүүлэх) боломжтой. Дараалалд `popleft()`, стекэд `pop()` ашигласан нь `deque`-ийн хоёр үзүүрийн үйлдэл тогтмол хугацаатай байдгаар зөвтгөгдөх бөгөөд жагсаалтад `pop(0)` хийвэл үлдсэн бүх элемент шилжих тул хугацаа элементийн тоотой өсдөг [13].

5.4. Зорилтод хүрсэн байдал

1.1 хэсэгт тавьсан зорилтууд дараах байдлаар биелсэн. Нийлмэл өгөгдлийн төрлүүдийг бодлогод тохируулан сонгосон нь үр дүнгээр харагдсан: давхардал арилгахад олонлог (4 жимсэнд 3 өвөрмөц өнгө), түлхүүрээр хайхад толь бичиг, өөрчлөгдөхгүй бичлэгт tuple, эрэмбэтэй өгөгдөлд жагсаалт ашиглагдсан. Файлыг `'r'`, `'w'`, `'a'` горимоор унших, бичих, нэмэх, устгах үйлдэл ажиллаж, мөр таслал, файл шинэчлэхтэй холбоотой онцлог илэрсэн. Мэдрэмжийн шинжилгээ 9 сэтгэгдлийн 7-г (77.8 %) зөв ангилж, үгүйсгэлийг ялгаж чаддаггүй хязгаарлалтаа харуулсан. Класс, удамшил, `deque` дээр суурилсан дараалал, стек хүлээгдсэнээр ажиллаж, цуцлалт, тэргүүлэх зэрэг, алдааны мэдээлэл нэмэгдсэн боловч эхэлсэн даалгаврыг цуцлах хэсэг хэрэгжээгүй.

6. ДҮГНЭЛТ

Лабораторийн ажлаар Python-ы нийлмэл өгөгдлийн төрлүүд, файлтай ажиллах горимууд, класс, удамшил, дараалал, стекийг ашигласан программууд бичиж туршсан. Үр дүнгээс дараах дүгнэлтүүд гарав:

1) Лаб 3.1-д tuple-ийн сондгой индекстэй сөрөг элементүүдийн үржвэр 36, тэгш ба сондгой тоонуудын жагсаалт (B буурах, C өсөх), 12 нэр томьёотой толь бичгийн хайлт, 10–100 хоорондох санамсаргүй тоо, 4 жимсэнд 3 өвөрмөц өнгө гарсан нь өгөгдлийн төрөл бүр өөр зорилгод тохирохыг харуулав.
2) Лаб 3.2-т `'w'` горим файлыг тэглэдэг, `'a'` горим төгсгөлд нэмдэг, `write()` шинэ мөрийг өөрөө нэмдэггүй болох нь тогтоогдсон. Түлхүүр үгэнд суурилсан мэдрэмжийн шинжилгээ 9 сэтгэгдлийн 7-г (77.8 %) зөв ангилсан бөгөөд «decent» үгийг эерэг жагсаалтад оруулснаас нэг, үгүйсгэл агуулсан «I am not happy with this service.» сэтгэгдэл нэг буруу ангилагдсан.
3) Лаб 3.3-т 4 даалгаврын нэг цуцлагдаж, нэг нь хоосон замаас болж бүтэлгүйтсэн ч дараалал үргэлжилж, үлдсэн нь тэргүүлэх зэргийн дагуу (T102, T104, T103) боловсруулагдсан. Ижил зэрэгтэй T102, T104 ирсэн дарааллаа хадгалсан нь `sorted()`-ийн тогтвортой байдлыг, `undo_last()` сүүлд орсон T103-ыг буцаасан нь стекийн LIFO зарчмыг харуулав.

Санал, зөвлөмж: мэдрэмжийн шинжилгээнд үгүйсгэх үгийн дараах үгийн утгыг эсрэгээр болгох дүрэм нэмэх, файлаас сэтгэгдэл устгасны дараа шинжилгээг дахин ажиллуулах, `cancel_task()`-д эхэлсэн даалгаврыг тэмдэглэх боломж нэмэх, оролтын утгыг шалгах

(`read(N)`-ийн N, толь бичгийн `strip()`), босго ба хоосон стекийн тохиолдлыг нэмж туршихыг зөвлөж байна.

7. АШИГЛАСАН МАТЕРИАЛ

[1] Лабораторийн ажил 3.1. Хиймэл оюун ухааны хэрэгслийн нэг хэсэг болох нийлмэл өгөгдлийн төрлүүдийг программчлалд ашиглах. «Хиймэл оюуны үндэс», Шинжлэх Ухаан Технологийн Их Сургууль, KOICA e-learning.

[2] Лабораторийн ажил 3.2. Хиймэл оюун ухааны программчлалд файлтай ажиллах. «Хиймэл оюуны үндэс», Шинжлэх Ухаан Технологийн Их Сургууль, KOICA e-learning.

[3] Лабораторийн ажил 3.3. Хиймэл оюун ухааны программчлалд класс, удамшил, дараалал, стекийг ашиглах. «Хиймэл оюуны үндэс», Шинжлэх Ухаан Технологийн Их Сургууль, KOICA e-learning.

[4] Лабораторийн тайлан бичих удирдамж. Шинжлэх Ухаан Технологийн Их Сургууль, Мэдээлэл холбооны технологийн сургууль.

[5] Python Software Foundation. Data Structures. The Python Tutorial. https://docs.python.org/3/tutorial/datastructures.html

[6] Python Software Foundation. random — Generate pseudo-random numbers. https://docs.python.org/3/library/random.html

[7] Python Software Foundation. Reading and Writing Files. The Python Tutorial. https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files

[8] Python Software Foundation. Built-in Functions: open(). https://docs.python.org/3/library/functions.html#open

[9] B. Liu. Sentiment Analysis and Opinion Mining. Morgan & Claypool Publishers, 2012.

[10] C. J. Hutto, E. Gilbert. VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text. Proceedings of the International AAAI Conference on Web and Social Media, 8(1), 2014, pp. 216–225. https://doi.org/10.1609/icwsm.v8i1.14550

[11] Python Software Foundation. Classes. The Python Tutorial. https://docs.python.org/3/tutorial/classes.html

[12] Python Software Foundation. Errors and Exceptions. The Python Tutorial. https://docs.python.org/3/tutorial/errors.html

[13] Python Software Foundation. collections — Container datatypes: deque objects. https://docs.python.org/3/library/collections.html#collections.deque

[14] Python Software Foundation. Sorting Techniques (Sorting HOWTO). https://docs.python.org/3/howto/sorting.html

8. ХАВСРАЛТ

Хавсралтад лабораторийн ажлын программын эх кодыг байрлуулав: ХАВСРАЛТ А — Лаб 3.1, ХАВСРАЛТ Б — Лаб 3.2, ХАВСРАЛТ В — Лаб 3.3.

ХАВСРАЛТ А. Лаб 3.1-ийн эх код

Файл: `lab3_1_1.py` (Бодлого 1)

lab3_1_1.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_1_1.py (3.14.4)

File Edit Format Run Options Window Help

```
numbers = (5, -2, 7, -3, 4, -6, 8, -1)

product = 1
found = False

for i in range(len(numbers)):
    if i % 2 == 1 and numbers[i] < 0:
        product *= numbers[i]
        found = True

if found:
    print("Сондгой индекстэй сөрөг элементүүдийн үржвэр:", product)
else:
    print("Ийм элемент олдсонгүй.")
```

Файл: `lab3_1_2.py` (Бодлого 2)

lab3_1_2.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_1_2.py (3.14.4)

File Edit Format Run Options Window Help

```
numbers = [12, 5, 8, 3, 17, 10, 6, 21, 4, 9]

B = []
C = []

for number in numbers:
    if number % 2 == 0:
        B.append(number)
    else:
        C.append(number)

B.sort(reverse=True)
C.sort()

print("Тэгш тоонууд B:", B)
print("Сондгой тоонууд C:", C)
```

Файл: `lab3_1_3.py` (Бодлого 3

lab3_1_3.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_1_3.py (3.14.4)

File Edit Format Run Options Window Help

```
ai_dictionary = {
    "хиймэл оюун ухаан": "Artificial Intelligence",
    "машин сургалт": "Machine Learning",
    "гүн сургалт": "Deep Learning",
    "нейрон сүлжээ": "Neural Network",
    "өгөгдөл": "Data",
    "алгоритм": "Algorithm",
    "загвар": "Model",
    "сургалт": "Training",
    "туршилт": "Testing",
    "таамаглал": "Prediction",
    "компьютерийн хараа": "Computer Vision",
    "байгалийн хэл боловсруулалт": "Natural Language Processing"
}

word = input("Монгол нэр томьёо оруулна уу: ").lower()

if word in ai_dictionary:
    print("Англи орчуулга:", ai_dictionary[word])
else:
    print("Тольд байхгүй байна.")
```

)

Файл: `lab3_1_4.py` (Бодлого 4)

lab3_1_4.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_1_4.py (3.14.4)

File Edit Format Run Options Window Help

```
import random

numbers = []

for i in range(10):
    numbers.append(random.randint(10, 100))

print("Санамсаргүй тоонууд:")
print(numbers)
```

Файл: `lab3_1_5.py` (Бодлого 5)

lab3_1_5.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_1_5.py (3.14.4)

File Edit Format Run Options Window Help

```
fruits = [
    {"name": "Apple", "price": 5000},
    {"name": "Banana", "price": 6000},
    {"name": "Orange", "price": 7000},
    {"name": "Mango", "price": 9000}
]

def find_fruits_by_price(price):
    result = []

    for fruit in fruits:
        if fruit["price"] == price:
            result.append(fruit["name"])

    return result

print("Жимснүүд:")
for fruit in fruits:
    print(fruit["name"], "-", fruit["price"], "₮")

price = int(input("\nYнэ оруулна уу: "))
result = find_fruits_by_price(price)

if result:
    print("Тухайн үнэтэй жимс:", result)
else:
    print("Тухайн үнэтэй жимс олдсонгүй.")

fruit_names = set()

for fruit in fruits:
    fruit_names.add(fruit["name"])

print("Жимсний set:", fruit_names)

fruit_info = ("Apple", 5000)
print("Tuple:", fruit_info)
```

ХАВСРАЛТ Б. Лаб 3.2-ын эх код

Файл: `lab3_2_jishee.py` (Жишээ 1)

lab3_3_jishee.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_3_jishee.py (3.14.4)

File Edit Format Run Options Window Help

```
# Import necessary libraries
with open("reviews.txt", "r") as file:
    reviews = file.readlines()  # Файлын бүх мөрийг уншина.

# Define simple keyword-based sentiment analysis
positive_words = ["love", "amazing", "fantastic", "recommend", "great"]
negative_words = ["worst", "disappointed", "not worth", "regret", "bad"]

results = []

for review in reviews:
    review_lower = review.lower()  # Том үсгийг жижиг үсгээр хөрвүүлнэ
    if any(word in review_lower for word in positive_words):
        sentiment = "Positive"
    elif any(word in review_lower for word in negative_words):
        sentiment = "Negative"
    else:
        sentiment = "Neutral"

    results.append(f"{review.strip()} -> Sentiment: {sentiment}")

# Үр дүнг шинэ файлд хадгална
with open("results.txt", "w") as output_file:
    for result in results:
        output_file.write(result + "\n")

print("Sentiment analysis complete! Check 'results.txt' for results.")
```

Файл: `lab3_2_jishee2.py` (Жишээ 2)

lab3_1_1.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_1_1.py (3.14.4)

File Edit Format Run Options Window Help

```
numbers = (5, -2, 7, -3, 4, -6, 8, -1)

product = 1
found = False

for i in range(len(numbers)):
    if i % 2 == 1 and numbers[i] < 0:
        product *= numbers[i]
        found = True

if found:
    print("Сондгой индекстэй сөрөг элементүүдийн үржвэр:", product)
else:
    print("Ийм элемент олдсонгүй.")
```

Файл: `lab3_2_jishee_3.py` (Жишээ 3)

lab3_2_jishee3.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_2_jishee3.py (3.14.4)

File Edit Format Run Options Window Help

```
import shutil
with open('file.txt', 'a') as f:
shutil.copyfileobj(source_file, f)
# with хэллэгийг ашиглан файлд өгөгдөл нэмэх арууды харуулах програм
L = ["This is Delhi \n", "This is Paris \n", "This is London \n"]
# Файлд бичих
with open("myfile.txt", "w") as file1:
# Writing data to a file
file1.write("Hello \n")
file1.writelines(L)
# Файлд бичих
with open("myfile.txt", 'a') as file1:
file1.write("Today")
# Reading from file
with open("myfile.txt", "r+") as file1:
# Файлаас унших
print(file1.read())
```

Файл: `lab3_2_1.py` (Бодлого 1)

lab3_2_1.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_2_1.py (3.14.4)

File Edit Format Run Options Window Help

```
reviews = [
    "I love this product, it's amazing!",
    "Worst experience ever, totally disappointed.",
    "The service was okay, nothing special.",
    "Absolutely fantastic! Highly recommend.",
    "Not worth the price, I regret buying it.",
    "Decent quality, but could be better."
]

with open("reviews.txt", "w", encoding="utf-8") as file:
    for review in reviews:
        file.write(review + "\n")

print("reviews.txt файл үүслээ.")
```

Файл: `lab3_2_jishee_4.py` (мэдрэмжийн шинжилгээний жишээ)

lab3_2_jishee4.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_2_jishee4.py (3.14.4)

File Edit Format Run Options Window Help

```
# Import necessary libraries
with open("reviews.txt", "r") as file:
    reviews = file.readlines()  # Файлын бүх мөрийг уншина.

# Define simple keyword-based sentiment analysis
positive_words = ["love", "amazing", "fantastic", "recommend", "great"]
negative_words = ["worst", "disappointed", "not worth", "regret", "bad"]

results = []

for review in reviews:
    review_lower = review.lower()  # Том үсгийг жижиг үсгээр хөрвүүлнэ
    if any(word in review_lower for word in positive_words):
        sentiment = "Positive"
    elif any(word in review_lower for word in negative_words):
        sentiment = "Negative"
    else:
        sentiment = "Neutral"

    results.append(f"{review.strip()} -> Sentiment: {sentiment}")

# Үр дүнг шинэ файлд хадгална
with open("results.txt", "w") as output_file:
    for result in results:
        output_file.write(result + "\n")

print("Sentiment analysis complete! Check 'results.txt' for results.")
```

Файл: `lab3_2_2.py` (Бодлого 2)

lab3_2_2.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_2_2.py (3.14.4)

File Edit Format Run Options Window Help

```
positive_words = [
    "good", "great", "excellent", "love", "best",
    "happy", "amazing", "fantastic", "recommend",
    "decent"
]

negative_words = [
    "bad", "terrible", "worst", "hate", "poor",
    "disappointed", "not worth", "regret"
]

with open("reviews.txt", "r", encoding="utf-8") as file:
    reviews = file.readlines()

results = []

for review in reviews:
    text = review.lower()
    positive_count = 0
    negative_count = 0

    for word in positive_words:
        if word in text:
            positive_count += 1

    for word in negative_words:
        if word in text:
            negative_count += 1

    if positive_count > negative_count:
        sentiment = "Positive"
    elif negative_count > positive_count:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"

    results.append((review.strip(), sentiment))

with open("results.txt", "w", encoding="utf-8") as file:
    for review, sentiment in results:
        file.write(review + " -> " + sentiment + "\n")

print("Sentiment analysis дууслаа.")
```

Файл: `lab3_2_3.py` (Бодлого 3)

lab3_2_3.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_2_3.py (3.14.4)

File Edit Format Run Options Window Help

```
with open("reviews.txt", "a", encoding="utf-8") as file:
    file.write("The product is amazing.\n")
    file.write("I am not happy with this service.\n")
    file.write("Good quality and fast delivery.\n")

print("Шинэ review-үүд нэмэгдлээ.")
```

Файл: `lab3_2_4.py` (Бодлого 4)

lab3_2_4.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_2_4.py (3.14.4)

File Edit Format Run Options Window Help

```
with open("reviews.txt", "r", encoding="utf-8") as file:
    reviews = file.readlines()

results = []

for review in reviews:
    text = review.lower()
    positive_count = sum(word in text for word in positive_words)
    negative_count = sum(word in text for word in negative_words)

    if positive_count > negative_count:
        sentiment = "Positive"
    elif negative_count > positive_count:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"

    results.append((review.strip(), sentiment))

with open("results.txt", "w", encoding="utf-8") as file:
    for review, sentiment in results:
        file.write(review + " -> " + sentiment + "\n")

print("Шинэ review-үүдийг дахин шинжилж дууслаа.")
```

Файл: `lab3_2_5.py` (Бодлого 5)

lab3_2_5.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_2_5.py (3.14.4)

File Edit Format Run Options Window Help

```
remove_review = "The service was okay, nothing special."

with open("reviews.txt", "r", encoding="utf-8") as file:
    reviews = file.readlines()

reviews = [
    review for review in reviews
    if review.strip() != remove_review
]

with open("reviews.txt", "w", encoding="utf-8") as file:
    file.writelines(reviews)

print("Сонгосон review устгагдлаа.")
```

Файл: `lab3_2_6.py` (Бодлого 6)

lab3_2_6.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_2_6.py (3.14.4)

File Edit Format Run Options Window Help

```
n = int(input("Хэдэн тэмдэгт унших вэ: "))

with open("results.txt", "r", encoding="utf-8") as file:
    content = file.read(n)

print("Уншсан хэсэг:")
print(content)
```

ХАВСРАЛТ В. Лаб 3.3-ын эх код

Файл: `lab3_3_1.py` (өөрчилсөн программ)

lab3_3_1.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_3_1.py (3.14.4)

File Edit Format Run Options Window Help

```
import heapq
from collections import deque


class Task:
    """Даалгаврын үндсэн класс (Base Task)"""

    def __init__(self, task_id, description, priority=1):
        self.task_id = task_id
        self.description = description
        self.priority = priority
        self.status = "Pending"

    def get_id(self):
        return self.task_id

    def get_priority(self):
        return self.priority

    def set_status(self, status):
        self.status = status

    def __str__(self):
        return f"[{self.task_id}] {self.description} | Status: {self.status} | Priority: {self.priority}"


class DataProcessingTask(Task):
    """Өгөгдөл боловсруулах даалгаврын SubClass"""

    def __init__(self, task_id, description, data_path, priority=1):
        super().__init__(task_id, description, priority)
        self.data_path = data_path

    def __str__(self):
        return super().__str__() + f" | Data: {self.data_path}"

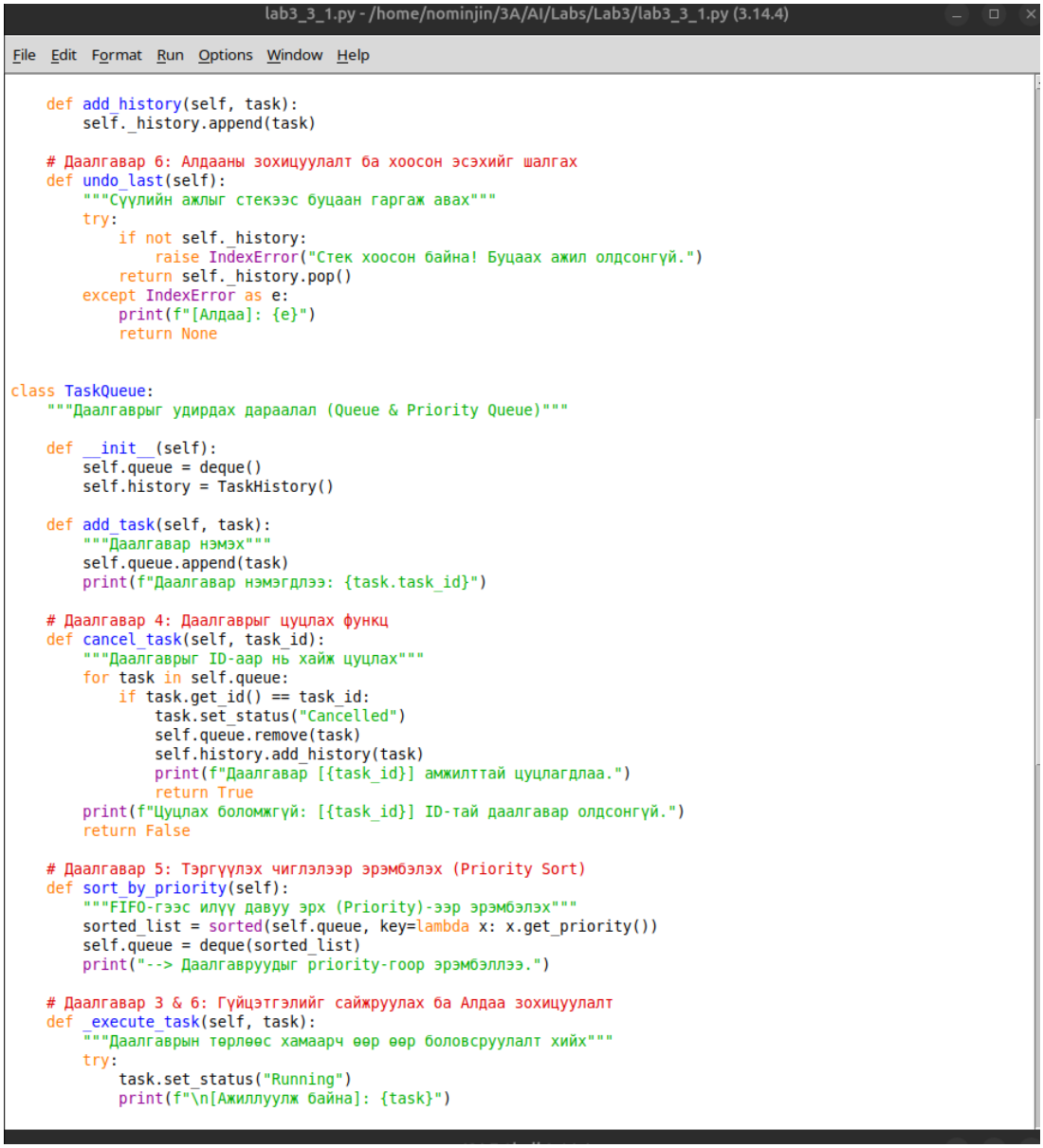

# Даалгавар 2: Шинэ subClass нэмсэн (ModelEvaluationTask)
class ModelEvaluationTask(Task):
    """Загварын үнэлгээний даалгаврын SubClass"""

    def __init__(self, task_id, description, model_name, accuracy_threshold, priority=1):
        super().__init__(task_id, description, priority)
        self.model_name = model_name
        self.accuracy_threshold = accuracy_threshold

    def __str__(self):
        return super().__str__() + f" | Model: {self.model_name} (Threshold: {self.accuracy_threshold})"


class TaskHistory:
    """Дууссан болон цуцлагдсан ажлуудыг хадгалах Стек (LIFO)"""

    def __init__(self):
        self._history = deque()
```


lab3_3_1.py - /home/nominjin/3A/AI/Labs/Lab3/lab3_3_1.py (3.14.4)
File Edit Format Run Options Window Help
def add_history(self, task):
self._history.append(task)
Даалгавар 6: Алдааны зохицуулалт ба хоосон эсэхийг шалгах
def undo_last(self):
"""Сүүлийн ажлыг стекээс буцаан гаргаж авах"""
try:
if not self._history:
raise IndexError("Стек хоосон байна! Буцаах ажил олдсонгүй.")
return self._history.pop()
except IndexError as e:
print(f"[Алдаа]: {e}")
return None
class TaskQueue:
"""Даалгаврыг удирдах дараалал (Queue & Priority Queue)"""
def __init__(self):
self.queue = deque()
self.history = TaskHistory()
def add_task(self, task):
"""Даалгавар нэмэх"""
self.queue.append(task)
print(f"Даалгавар нэмэгдлээ: {task.task_id}")
Даалгавар 4: Даалгаврыг цуцлах функц
def cancel_task(self, task_id):
"""Даалгаврыг ID-аар нь хайж цуцлах"""
for task in self.queue:
if task.get_id() == task_id:
task.set_status("Cancelled")
self.queue.remove(task)
self.history.add_history(task)
print(f"Даалгавар [{task_id}] амжилттай цуцлагдлаа.")
return True
print(f"Цуцлах боломжгүй: [{task_id}] ID-тай даалгавар олдсонгүй.")
return False
Даалгавар 5: Тэргүүлэх чиглэлээр эрэмбэлэх (Priority Sort)
def sort_by_priority(self):
"""FIFO-гээс илүү давуу эрх (Priority)-ээр эрэмбэлэх"""
sorted_list = sorted(self.queue, key=lambda x: x.get_priority())
self.queue = deque(sorted_list)
print("--> Даалгавруудыг priority-гоор эрэмбэллээ.")
Даалгавар 3 & 6: Гүйцэтгэлийг сайжруулах ба Алдаа зохицуулалт
def _execute_task(self, task):
"""Даалгаврын төрлөөс хамаарч өөр өөр боловсруулалт хийх"""
try:
task.set_status("Running")
print(f"\n[Ажиллуулж байна]: {task}")

```
            if isinstance(task, DataProcessingTask):
                if not task.data_path:
                    raise ValueError("Файлын зам олдсонгүй!")
                print(f"  -> '{task.data_path}' файл дээр өгөгдлийн цэвэрлэгээ ба боловсруулалт хийж байна...")

            elif isinstance(task, ModelEvaluationTask):
                if task.accuracy_threshold < 0 or task.accuracy_threshold > 1:
                    raise ValueError("Нөхцөлт нарийвчлалын утга 0-1 хооронд байх ёстой!")
                print(f"  -> '{task.model_name}' загварыг {task.accuracy_threshold} босгоор үнэлж байна...")

            else:
                print("  -> Ерөнхий даалгаврыг ажиллуулж байна...")

            task.set_status("Completed")
            self.history.add_history(task)
            print(f"[Амжилттай дууслаа]: {task.task_id}")

        except Exception as e:
            task.set_status("Failed")
            print(f"[Алдаа гарлаа] Даалгавар {task.task_id} бүтэлгүйтлээ! Шалтгаан: {e}")

    def process_all_tasks(self):
        """Дараалалд байгаа бүх даалгавруудыг боловсруулах"""
        print("\n--- Даалгавруудыг ажиллуулж эхэллээ ---")
        while self.queue:
            task = self.queue.popleft()
            self._execute_task(task)


# ==========================================
# Тест хийх ба Код ажиллуулах хэсэг
# ==========================================
if __name__ == "__main__":
    t_queue = TaskQueue()

    # Даалгавруудыг үүсгэн нэмэх
    task1 = Task("T101", "Лог файл цэвэрлэх", priority=3)
    task2 = DataProcessingTask("T102", "Датасет бэлтгэх", "data/reviews.csv", priority=1)
    task3 = ModelEvaluationTask("T103", "LLM Загвар үнэлэх", "GPT-4o", 0.85, priority=2)
    task4 = DataProcessingTask("T104", "Алдаатай даалгавар тест", "", priority=1)  # Алдаа заах ёстой (Empty path

    t_queue.add_task(task1)
    t_queue.add_task(task2)
    t_queue.add_task(task3)
    t_queue.add_task(task4)

    # Даалгавар цуцлах тест
    t_queue.cancel_task("T101")

    # Тэргүүлэх чиглэлээр эрэмбэлэх тест
    t_queue.sort_by_priority()

    # Даалгавруудыг ажиллуулах
    t_queue.process_all_tasks()

    # Стекээс Undo хийж буцаах тест
    print("\n--- Стекийн тест (Undo) ---")
    last_task = t_queue.history.undo_last()
    if last_task:
        print(f"Сүүлд хийгдсэн ажил: {last_task}")
```